

RAICHU: Rendering Acceleration Integrating CGRA arcHitecture for Unified VR Pipelines

Yanzhou Tang*
New York University
New York, NY, USA
ytang614@gatech.edu

Haiyu Wang
New York University
New York, NY, USA
hw3689@nyu.edu

Cheng Tan
Google/ASU
Mountain View, CA, USA
chengtan@google.com

Sai Qian Zhang
New York University
New York, NY, USA
sai.zhang@nyu.edu

Abstract—Virtual reality (VR) systems often execute multiple tasks concurrently, among which rendering typically dominates overall execution time. This computational dominance poses a fundamental challenge, as the system must simultaneously deliver high visual fidelity and maintain real-time frame rates, both essential for achieving responsive and immersive user interaction. Ray tracing and neural rendering such as Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS) each offer distinct tradeoffs between performance and fidelity, which in turn motivates the shift towards hybrid rendering in modern VR pipelines. However, the diverse workloads within these pipelines stress different resources, and typical edge devices lack the architectural capability to support all of them efficiently.

We present RAICHU, a Rendering Acceleration Integrating CGRA arcHitecture for Unified VR Pipelines that directly addresses this challenge via a comprehensive software-hardware co-design. At the software level, a reconfigurable dataflow toolchain maps these heterogeneous kernels onto the CGRA and optimizes their dataflow. At the hardware level, a heterogeneous CGRA is co-designed with the compiler, addressing the inherent limitations of dataflow architectures in handling memory bound tasks. To our knowledge, RAICHU is the first rendering framework to accelerate multiple rendering pipelines within a single architecture. Evaluation shows up to $109.65\times$ higher performance than NVIDIA Jetson Orin NX and $12.33\times$ better energy efficiency than other unified rendering accelerators.

Index Terms—Virtual Reality, Rendering, 3DGS, NeRF, Ray Tracing, Coarse Grained Reconfigurable Arrays

I. INTRODUCTION

Virtual Reality (VR) is rapidly evolving from an entertainment technology into a key next-generation computing platform. As display technologies advance towards higher resolutions and refresh rates, the boundary between physical and virtual worlds becomes increasingly seamless. This evolution demands highly realistic and responsive VR experiences to ensure user immersion, which places massive compute and memory demands on the underlying system.

Among the diverse components of a VR system, rendering remains the primary bottleneck and the critical determinant of perceptual quality. Fundamentally, rendering simulates light transport to generate realistic imagery. As illustrated in Figure 1, the process begins with ray marching from the virtual camera, where rays interact with the scene’s spatial representation, such as Multilayer Perceptrons (MLPs) in NeRF [1], explicit

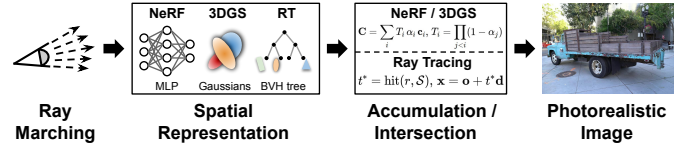


Fig. 1. A rendering pipeline [4], [5] where rays interact with spatial representations such as MLPs, Gaussians, and BVHs to produce photorealistic images.

Gaussians in 3DGS [2], or Bounding Volume Hierarchies (BVH) in ray tracing. These interactions are then processed through accumulation or intersection tests to compute the final pixel color. The sense of immersion hinges on achieving photorealistic imagery while adhering to hard real-time constraints, typically ranges from 50–70 ms [3]. This dual requirement creates a fundamental tension: advanced rendering algorithms capable of photorealism often exceed the tight timing and power budgets of standalone edge VR devices. Moreover, the rendering pipeline must share limited on-device compute and memory with other latency-sensitive tasks such as Simultaneous Localization and Mapping (SLAM), eye tracking, and Asynchronous Timewarp (ATW), leading to severe resource contention.

Beyond the challenge of resource contention, the rendering task itself presents a multi-faceted optimization problem. It must satisfy the concurrent demands for high visual quality, real time frame rates, and memory efficiency on resource constrained edge devices. As summarized in Table I, different rendering methods exhibit a complex trade-off space across these three metrics. For instance, classic NeRF [1] offer high memory efficiency but suffer from poor rendering quality and slow performance. Newer variants such as Hash Grid NeRF [6] and Low Rank NeRF [7] improve quality and memory efficiency but still suffer from low rendering speed. Conversely, 3DGS [2] delivers both high quality and fast rendering, though its large memory footprint limits edge deployment. Finally, traditional ray tracing, while offering the highest visual fidelity, incurs the worst performance and memory efficiency.

The distinct tradeoffs of these rendering methods, as summarized in Table I, naturally make them suitable for different roles within a complete VR scene. For instance, the high memory efficiency of NeRF is ideal for representing large,

*Also with Georgia Institute of Technology, work done during internship at New York University.

TABLE I
COMPARATIVE OVERVIEW OF RENDERING METHODS. STATIC BG
DENOTES STATIC BACKGROUND, FG OBJECTS DENOTES FOREGROUND
OBJECTS, AND MEM. EFF. REFERS TO MEMORY EFFICIENCY.

Attributes	M-NeRF	H-NeRF	L-NeRF	3DGS	RT
Speed	Medium	High	High	High	Low
Quality	Medium	High	High	High	Very High
Mem. Eff.	Very High	High	High	Medium	Medium
Typical Use Case	Legacy Low-End	Static BG	Static BG	FG Objects Avatars	Dynamic Effects
Key Refs	[1]	[6]	[7]	[2]	[12], [13] [14]

Note: M-NeRF denotes MLP-based NeRF; H-NeRF denotes Hash Grid NeRF; L-NeRF denotes Low-Rank Decomposition NeRF; RT denotes ray tracing.

static backgrounds [6], [8], while the high speed and quality of 3DGS are well-suited for dynamic foreground objects or avatars [9], [10]. Selective ray tracing remains necessary for capturing physically accurate effects like transparent surfaces or hard shadows [11]. This motivates the need for a unified accelerator capable of efficiently processing all these diverse modalities, a capability that current edge architectures lack.

Building on these insights, we introduce *RAICHU* for Rendering Acceleration Integrating CGRA arcHitecture for Unified VR Pipelines. RAICHU presents a unified framework that accelerates diverse rendering algorithms via an integrated software–hardware co-design approach. To the best of our knowledge, RAICHU is the **first framework capable of efficiently accelerating NeRF, 3DGS, and Ray Tracing within a single, cohesive edge architecture**. The key contributions of this work are summarized as follows:

- We propose a software–hardware co-design for unified VR rendering acceleration using a plug-in CGRA accelerator. Our compiler applies constant folding and operation fusion to shrink the data flow graph, improving CGRA mapping efficiency and data reuse.
- We implement a plug-in CGRA accelerator within the VR SoC, featuring a memory CGRA that decouples computation from memory access and delivers structured data streams to the PE array. This design maximizes PE utilization and fully exploits the CGRA’s capability across diverse rendering workloads.
- We validate RAICHU through a user study demonstrating no perceptual quality degradation, and our evaluations across NeRF, 3DGS, and ray tracing show that RAICHU outperforms edge GPUs and prior unified rendering accelerators in both performance and energy efficiency.
- We introduce an end to end framework enabling seamless mapping from high level Python code to the heterogeneous reconfigurable fabric. Our evaluation demonstrates that RAICHU achieves up to $109.7\times$ speedup over the Orin NX GPU, $6.1\times$ over state of the art unified accelerators, and $13.1\times$ over domain specific accelerators.

II. BACKGROUND AND RELATED WORK

A. VR System

A typical standalone VR headset is built upon a heterogeneous System on Chip (SoC) architecture. This system integrates several specialized processing engines including a CPU, GPU, NPU, and a media processing unit (MPU). These components are interconnected via a shared Network on Chip (NoC) to a unified DRAM. High speed peripherals interface directly with this fabric, where image sensors stream data via MIPI CSI [15] to enable real time environment tracking and low latency interaction.

The CPU, built on a multi core architecture, manages system control, task scheduling, and peripheral coordination, directing data and workloads across heterogeneous processors. The GPU, with many streaming multiprocessors, handles geometry processing, shading, and pixel level computation, forming the backbone for real time stereoscopic rendering and scene reconstruction. The NPU focuses on machine learning inference for tasks such as tracking, SLAM, and foveated rendering, reducing latency and power by offloading AI workloads from the CPU and GPU. The MPU integrates specialized accelerators for latency critical sensing and media.

The end to end VR workflow forms a tightly coupled pipeline (*Capture* $\rightarrow$ *Perceive* $\rightarrow$ *Render* $\rightarrow$ *Display*). Sensors deliver data through MIPI, the NPU performs spatial and semantic inference, the GPU handles graphics generation, and the CPU manages scheduling and timing to maintain data consistency across modules. This coordinated use of heterogeneous processors enables low latency and energy efficient immersive rendering.

B. Coarse-Grained Reconfigurable Architecture

Coarse Grained Reconfigurable Arrays (CGRAs) are spatial dataflow architectures that balance the flexibility of general purpose processors with the energy efficiency of ASICs [16]. Unlike bit level FPGAs, CGRAs use word level processing elements with ALUs, multipliers, and local registers connected through a reconfigurable network. They exploit loop level parallelism by converting compute intensive loops into dataflow graphs that are mapped onto the PE array using modulo scheduling [17], [18], enabling spatial execution that avoids the von Neumann fetch and decode bottleneck and allows direct data movement between producer and consumer PEs. CGRAs offer high energy efficiency and flexibility, making them effective for regular streaming workloads and compute dense kernels. Originating in DSP [19] and extended to multimedia tasks such as video and image processing [20], CGRAs have more recently been used for energy efficient DNN inference by adapting to dense tensor operations, sparsity, and diverse operator patterns [21], [22].

C. Rendering Methods

Next, we summarize the principles of NeRF, 3DGS, and ray tracing, the three major rendering methods supported by RAICHU.

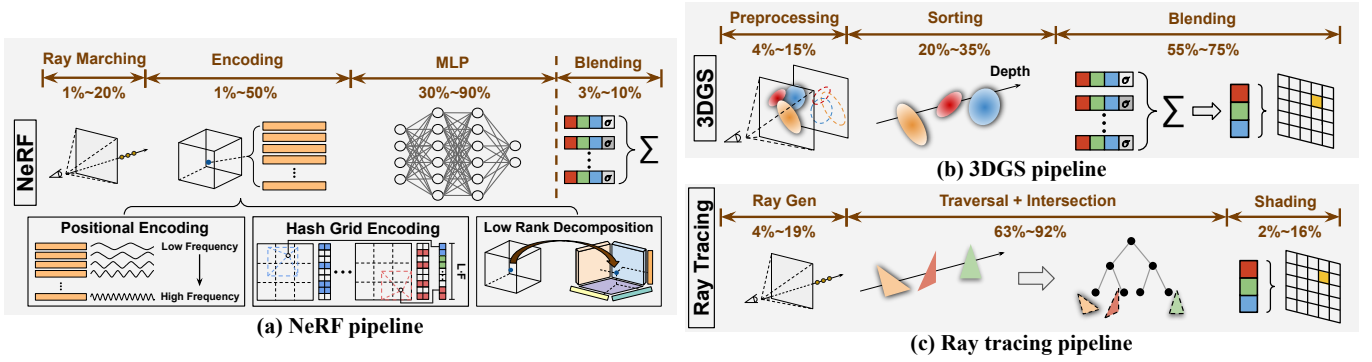


Fig. 2. (a) NeRF pipeline breakdown. (b) 3DGS pipeline breakdown. (c) Ray tracing pipeline breakdown.

As shown in Figure 2a, the NeRF rendering pipeline consists of several main stages. It begins with *Ray Marching*, where rays are sent out from the camera and a sequence of 3D sample points is generated along each ray. The next stage is *Encoding*, which transforms the low-dimensional coordinates of these points into high-dimensional feature vectors so the network can capture fine detail. These encoded features then flow into a Multi Layer Perceptron (MLP), which predicts the volume density (σ) and the view dependent RGB color for every sampled point. The final stage is *Ray Composition*, where density and color predictions along each ray are accumulated through volume rendering to determine the pixel value. NeRF variants differ mainly in their encoding schemes. The original NeRF uses Positional Encoding, while newer approaches such as hash grid NeRF (e.g., Instant NGP [6]) and low rank NeRF (e.g., TensorRF [7]) rely on Hash Encoding and Low Rank Decomposition. As indicated by the latency breakdown in Figure 3, although these newer encodings improve overall rendering speed, they also make the encoding stage a new performance bottleneck.

The pipeline for 3D Gaussian Splatting (3DGS), shown in Figure 2b, follows a rasterization based approach. It begins with a *Preprocessing* stage, where 3D Gaussian primitives are projected onto the 2D view plane. This step calculates their 2D covariance and screen space extent. The pipeline then proceeds to a *Sorting* stage that orders the projected splats by depth to ensure correct alpha blending and occlusion. The final stage is *Blending*, in which the sorted splats are drawn onto the pixel grid. Each splat contributes color and opacity to the pixels it overlaps, and these contributions are accumulated to form the final image. Among all stages, Blending has the highest latency and serves as the main computational bottleneck in the 3DGS pipeline.

The traditional ray tracing pipeline consists of three main stages. It begins with *Ray Generation*, where a primary ray is cast from each pixel in the camera view into the 3D scene. The next stage is *BVH Traversal* and *Intersection Test*, in which each ray travels through a Bounding Volume Hierarchy to locate the closest point of intersection with scene geometry. The final stage is *Shading*, where the color at the intersection point is computed, often by casting additional rays

for shadows or reflections and evaluating the material and lighting conditions. Among these stages, BVH Traversal and Intersection Test is the most computationally demanding and therefore serves as the primary bottleneck in the pipeline.

D. Accelerators for Rendering

Hardware accelerators have been designed to support rendering techniques such as NeRF, 3DGS, and ray tracing, each targeting the distinct bottlenecks of its respective method. NeRF accelerators focus on memory throughput challenges [23]–[29], including resolving bank conflicts introduced by hash encoding, and exploit sparsity by skipping computations for sample points that have minimal influence on the final image [30], [31]. Work on 3DGS targets its rasterization pipeline, including pruning Gaussian primitives [32]–[34] and optimizing the high bandwidth sorting stage required for alpha blending [32], [35] through improved memory access patterns and on chip buffering. Ray tracing acceleration focuses on reducing the cost of BVH traversal and intersection tests, using improved BVH structures [14], [36], [37], mitigating thread imbalance [13], and applying prediction mechanisms [38]. More recently, unified accelerators have begun to appear for neural rendering workloads such as NeRF and 3DGS [39], [40], leveraging shared computational structure to provide flexible and efficient hardware support across multiple rendering paradigms. However, most existing accelerators rely on fixed-function hardware pipelines designed for specific bottlenecks, and they lack a programmable framework for user-level configuration or pipeline adaptation. As a result, these architectures are limited to predefined execution flows and cannot easily support custom changes or evolving algorithmic structures.

III. MOTIVATION

A. Rendering Performance Profiling in Multitask VR System

Modern VR systems concurrently execute multiple latency sensitive tasks, such as SLAM [41], eye tracking [42], and asynchronous timewarp [43], alongside the rendering workload. As shown in Figure 3a, we profile six concurrent multitasking scenarios on a Jetson Orin NX based VR prototype, a platform widely used in prior work to emulate VR devices [39], [44], [45]. Our results show rendering contributes

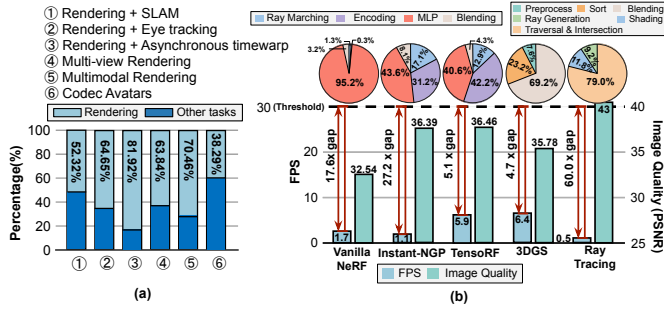


Fig. 3. (a) Workload characterization of parallel applications in VR system. (b) Performance, image quality, and latency breakdown of rendering methods on an edge GPU.

the largest share of end to end latency, emerging as a primary system bottleneck, often preventing the device from meeting real time deadlines and causing increased motion-to-photon delay [46], [47] and visual stutter [48]. Given the importance of rendering and its computational demand, we believe that offloading this workload to a dedicated accelerator can reduce GPU pressure and enable other latency critical tasks to run more efficiently.

Figure 3b compares the performance and image quality of several rendering methods on an Jetson Orin. The 3DGS and NeRF variants are evaluated on the NeRF Synthetic dataset [1], and ray tracing is measured using LumiBench [49]. Ray tracing delivers the highest image quality at 43 PSNR but does so at a prohibitive cost, reaching only 0.5 frames-per-seconds (FPS). Neural rendering methods such as 3DGS and NeRF achieve much higher frame rates, with 3DGS reaching 6.4 FPS, but at the expense of peak fidelity. 3DGS also carries a large memory footprint, often several hundred megabytes, which poses challenges for deployment on resource limited edge devices. These results highlight the complementary strengths of the three methods: ray tracing provides the highest fidelity for fine details, whereas 3DGS and NeRF are better suited for backgrounds or regions with lower precision requirements, consistent with observations from prior work [50], [51]. To better support diverse applications with different visual priorities, a versatile VR system should be able to accelerate all of these rendering approaches and enable dynamic, region specific rendering policies, since each method has its own bottlenecks. For example, a foveated rendering system can employ high fidelity ray tracing at the gaze center while using NeRF or 3DGS for peripheral regions to maintain high overall frame rates. Similarly, a complex VR scene might represent vast static environments with NeRF but render interactive avatars using 3DGS to ensure low latency responsiveness.

Additionally, while Figure 3b demonstrates the potential of these rendering techniques, it also reveals a key limitation: none of them achieve real time performance on the edge GPU, which prior work suggests is 72–90 FPS range commonly targeted by interactive VR systems [52]–[54]. This gap further motivates the need for specialized acceleration. Yet, as illustrated in Figure 3b, building an efficient accelerator

TABLE II
COMPARISON OF CGRA AND OTHER ARCHITECTURES

	CPU	GPU	FPGA	CGRA	ASIC
Energy Eff.	Low	Medium	High	High	Very High
Power	High	Very High	Medium	Low	Low
Flexibility	Very High	High	High	Medium	Low

is nontrivial because each rendering method has a fundamentally different bottleneck. Vanilla NeRF is dominated by its MLP, which accounts for 95.2% of total latency. Grid-based methods such as Instant NGP and TensorRF distribute their time between encoding (31.2% and 42.2%) and the MLP (43.6% and 40.6%). The 3DGS pipeline is limited by sorting (23.2%) and blending (69.2%), whereas conventional ray tracing is overwhelmingly bottlenecked by BVH traversal and intersection (79.0%). This pronounced diversity indicates that **an accelerator tailored to only one rendering paradigm would be ineffective for the others**, making a unified accelerator architecture essential for supporting the full spectrum of rendering workloads.

B. Why CGRA?

As shown in Table II, CGRAs combine the efficiency of specialized hardware with the programmability of processors, making them well-suited for the evolving rendering demands of VR edge devices [16], [55]. Standalone VR systems operate under tight power and thermal budgets, and rendering is both compute-intensive and power-hungry, requiring hardware that delivers high energy efficiency at low absolute power. Owing to their efficiency, CGRAs have been widely adopted in IoT and edge-class applications, with some designs achieving sub-milliwatt power for ultra-low-energy workloads [56], [57]. Leveraging this architecture as a rendering accelerator is therefore highly promising.

Programmability is equally important because VR rendering is rapidly evolving, with methods such as NeRF and 3DGS introducing diverse and shifting computational patterns. CGRAs naturally support this variability through PE-level reconfigurability, enabling efficient execution across heterogeneous algorithms [16], [58]. Finally, while CGRAs provide general flexibility, they also allow domain-specific customization. By examining shared structures and bottlenecks across rendering methods, the CGRA can be tuned to optimize common kernels, achieving a balanced design that offers both programmability and specialized performance.

C. Common Patterns Between Rendering Methods

NeRF, 3DGS, and ray tracing rely on different scene representations, but they all follow a broadly similar pipeline structure. In the front end, each ray or sample gathers a small amount of data through highly irregular memory accesses. In the back end, intensive computation is performed for intersection, neural inference, or composition. As a result, the front end is often bottlenecked by memory access, while the back end offers substantial parallelism that is not fully utilized. The

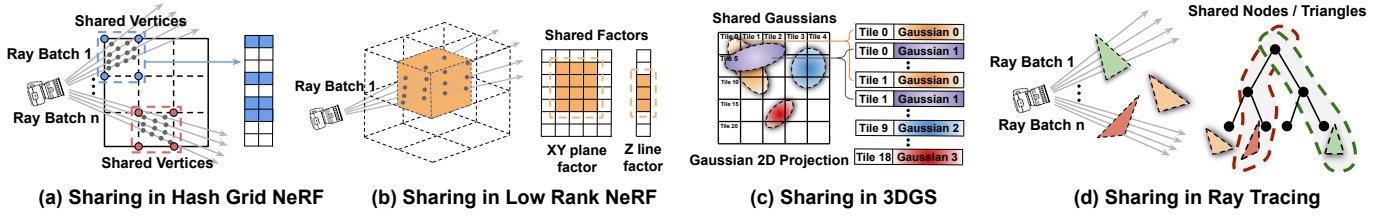


Fig. 4. Common data sharing patterns across modern rendering methods. (a) In Hash Grid NeRF, rays within the same batch query highly shared sets of hash grid vertices. (b) In Low Rank NeRF, samples repeatedly reuse localized patches of plane factors and contiguous segments of line factors. (c) In 3DGS, projected Gaussians are reused by many pixels and often by multiple tiles. (d) In Ray Tracing, rays in a batch traverse largely the same BVH nodes and hit shared triangles.

key insight, however, is that nearby rays tend to access many of the same primitives repeatedly. This access irregularity is therefore not arbitrary; instead, there is considerable latent sharing across samples.

1) *Hash Grid NeRF*: As illustrated in Figure 4a, in Hash Grid NeRF, samples from nearby rays often query nearly identical grid vertices across multiple levels, leading to reuse of the same vertex features as well as the same interpolation and neural computations.

2) *Low Rank NeRF*: As illustrated in Figure 4b, in Low Rank NeRF, samples within a ray group typically lie in a compact region, corresponding to small patches on the plane factors and short segments on the line factors. As a result, many samples reuse the same factor vectors and differ only in their interpolation weights.

3) *3DGS*: As shown in Figure 4c, each Gaussian typically covers a block of pixels and often extends across multiple tiles. Consequently, the same Gaussian appears in many tile lists, causing its parameters and projected conic to be loaded and evaluated repeatedly. This results in redundant memory traffic and repeated blending computations.

4) *Ray Tracing*: As illustrated in Figure 4d, during ray tracing, nearby rays often traverse highly similar paths through the bounding volume hierarchy. They visit many of the same internal nodes and frequently intersect the same triangles. However, conventional traversal is still performed on a per ray basis, leading to repeated node loads and redundant intersection computations.

Building on these observations, reshaping irregular accesses into shared batches allows the compute stage to better match the strengths of a coarse grained reconfigurable array, enabling its parallel units to be utilized much more effectively. This insight suggests a decoupled architecture that separates memory orchestration from parallel computation into different pipeline stages. By isolating the collection of shared primitives from the main computation, the hardware can operate in a streaming manner. This enables a macro-pipeline in which the system prepares and reshapes the next data batch while the current batch is still being processed. Such producer-consumer coordination hides the latency of irregular memory accesses and turns the traditionally serial rendering process into a high-throughput streaming workflow.

D. Design Insights, Tradeoffs, and Challenges

The observations above reveal the key insight behind RAICHU: despite their different scene representations, NeRF variants, 3D Gaussian Splatting, and ray tracing share a memory-compute execution structure. Their front-end stages perform irregular but shareable accesses to spatial primitives, while their back-end stages consume these primitives through dense parallel computation. Based on this insight, RAICHU makes three key design choices. First, it decouples memory orchestration and arithmetic computation using a heterogeneous **M-CGRA/C-CGRA** organization. Second, it employs the **Data Access Engine** (DAE) and shared buffers to bridge irregular memory accesses and regular compute execution. Third, it co-optimizes the compiler and architecture to efficiently map rendering pipelines onto the heterogeneous substrate.

This design makes an explicit tradeoff. RAICHU is not a general-purpose CGRA for arbitrary workloads; instead, it specializes the reconfigurable fabric for rendering memory-compute pipelines. Compared with fixed-function accelerators, RAICHU preserves programmability across NeRF variants, 3DGS, ray tracing, and related pipelines; compared with conventional CGRAs, it sacrifices some generality for higher efficiency on this workload class.

This specialization shifts the main design challenge to compilation. The compiler must partition high-level rendering kernels across two heterogeneous CGRAs, lower irregular accesses into M-CGRA primitives, fuse arithmetic patterns for the C-CGRA, and generate the shared buffer layout that connects the two lowering paths. Section IV describes how RAICHU addresses this challenge through hardware-software co-design.

IV. METHODOLOGY

The core idea behind RAICHU is a unified and reconfigurable framework that supports multiple rendering pipelines, including NeRF, 3DGS, and ray tracing. As discussed in Section III-C, although these workloads differ in mathematical formulation, they share a common structure: dense computation combined with irregular, memory-bound dataflow. RAICHU exploits this commonality through a tightly integrated hardware-software co-design. In this section, we de-

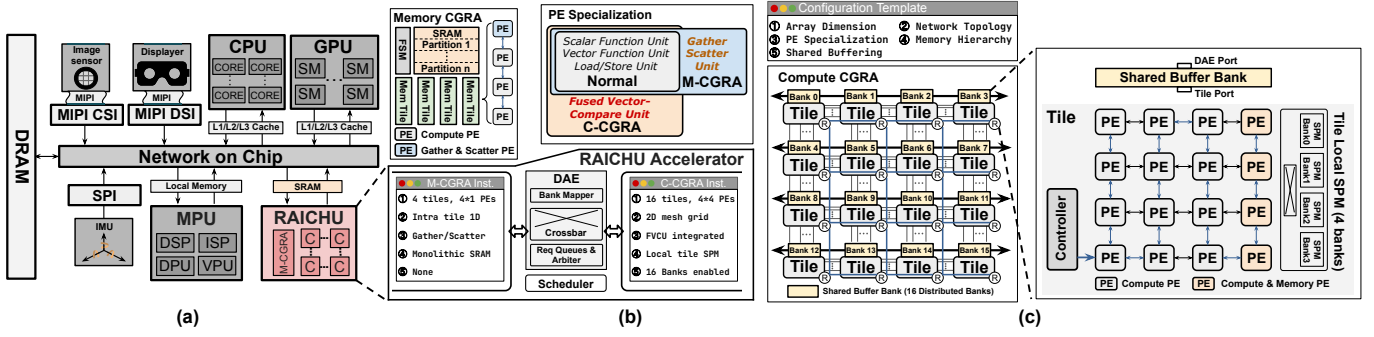


Fig. 5. (a) VR SoC with RAICHU accelerator. (b) RAICHU accelerator microarchitecture. (c) Tile microarchitecture.

scribe the RAICHU system architecture, compiler support, and mapping workflow.

A. Hardware Architecture Overview

RAICHU operates as a plug-in unified rendering accelerator integrated into a heterogeneous VR SoC, as shown in Figure 5a. The SoC CPU controls RAICHU through a scheduler interface. While CPU, GPU, and MPU subsystems collectively manage tracking, display, and general purpose computation, RAICHU offloads the rendering kernels that dominate frame latency and exhibit strong structural regularities. RAICHU does not alter the existing VR pipeline, but replaces the GPU execution of supported rendering kernels with accelerator execution.

To support this role, RAICHU adopts a dual-tier reconfigurable architecture. It consists of a **Memory CGRA** (M-CGRA) for programmable data orchestration and filtering, and a **Compute CGRA** (C-CGRA) for executing batched, regularized kernels.

The methodological core of RAICHU is a unified tile template that supports the creation of diverse reconfigurable fabrics. By freely selecting configuration parameters such as array dimension, network topology, PE specialization, memory hierarchy, and shared buffering, we can realize different types and scales of CGRA instances to meet specific application requirements. Figure 5b illustrates the full RAICHU accelerator featuring a dual tier fabric composed of these specialized instances.

As illustrated in Figure 5b, M-CGRA is the first tier of the fabric, designed to manage irregular data movement directly on the DRAM path. It is instantiated from the unified template with four memory tiles. Each tile is configured as a 1 by 4 vertical PE array with linear interconnects between adjacent elements. Unlike a distributed memory setup, the M-CGRA features a large monolithic SRAM shared across all memory tiles. This centralized buffer sits parallel to the tiles, providing a vast contiguous space for complex data reorganization and orchestration tasks.

As illustrated in Figure 8, the M-CGRA features a four-stage programmable pipeline optimized for decoupled address generation and data gathering. While the lower three PEs use

general-purpose ALUs for algorithm-specific address computations, the topmost PE integrates a Gather/Scatter (GA) unit to collect shared primitives. Through a 32×32 global crossbar, the four M-Tiles connect to a 32-bank shared SRAM. This enables each GA unit to simultaneously issue 8-wide requests to multiple banks, achieving a peak aggregate bandwidth of 32 data elements per cycle. Hardware arbiters transparently serialize any bank conflicts. This vertical topology and multi-banked SRAM reshape irregular memory fetches into dense data batches, effectively masking memory latency via its deep pipeline and highly concurrent memory subsystem.

The C-CGRA is the second tier of the fabric, instantiated from the unified template as a 4×4 grid of tiles. As shown in Figure 5c, each tile is configured with a 4 by 4 PE array, totaling 256 processing elements across the entire fabric. The network topology is established as a 2D mesh grid to support high density data movement and spatial reuse during rendering. In terms of memory hierarchy, each tile contains a multi banked tile local scratchpad memory (SPM) for near data computation.

Furthermore, the C-CGRA enables the shared buffering knob by connecting each tile to a corresponding bank of the shared buffer system. These banks feature dual ports to mediate streaming dataflow from the M-CGRA and local access from the tile itself. The internal PE specialization of the C-CGRA is tailored for intensive rendering arithmetic. As detailed in Figure 7, each PE features specialized functional units including scalar units, vector units, and a fused vector compare unit (FVCU). The FVCU is specifically designed to execute core arithmetic and geometric primitives common across modern rendering pipelines. This organization allows RAICHU to maintain tightly coupled dataflow between the M-CGRA driven preprocessing and tile level computation.

The Data Access Engine (DAE) manages the communication between the orchestration tier and the compute tier. The processed data from the M-CGRA is placed into a distributed shared buffer system consisting of 16 banks. A lightweight crossbar within the DAE ensures efficient routing based on bank locality and dependency constraints. This partitioned architecture creates a macro pipeline where the M-CGRA proactively prepares and reshapes data while the compute tier

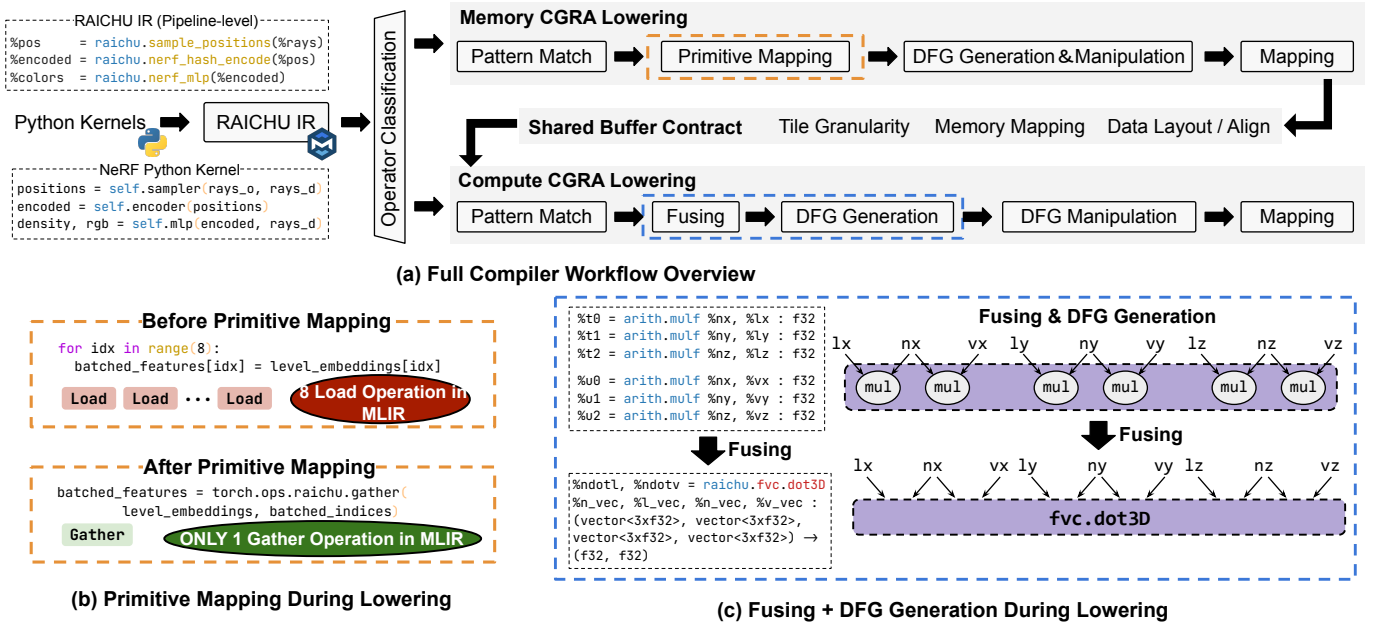


Fig. 6. (a) E2E compiler workflow. (b) M-CGRA side primitive mapping. (c) C-CGRA side operator fusing and DFG generation.

simultaneously processes the current workload. This streaming synergy effectively hides the long latency of irregular DRAM access and ensures that the hardware remains highly utilized across diverse rendering modalities.

B. Compiler and Programming Model

The RAICHU compiler translates high level rendering kernels into representations that can be executed efficiently by the M-CGRA and C-CGRA. As Figure 6a shows, the process begins by lowering Python kernels into RAICHU IR, a pipeline oriented form where major rendering stages such as hash grid encoding, Gaussian accumulation, MLP and intersection test appear as explicit operations. This level preserves the structure of the rendering pipeline without binding computation to hardware.

Next, the compiler classifies operators according to their algorithmic properties and user specified configuration. Stages with irregular data access or memory dominated behavior are mapped to the M-CGRA, while compute-intensive regions that benefit from parallel arithmetic are assigned to the C-CGRA. As a result, the RAICHU IR naturally serves as the point where the rendering pipeline splits into two coordinated lowering paths.

1) **Co-design with Memory CGRA:** The M-CGRA manages irregular data access patterns prevalent in rendering pipelines. To support this specialized unit, the compiler follows a structured workflow comprising Pattern Matching, Primitive Mapping, Dataflow graphs (DFG) Generation, DFG Manipulation, and Mapping. While this sequence mirrors the compute path, the primitive mapping stage is uniquely critical for translating high level algorithmic intent into efficient hardware memory operations. The overall position of this process within the compiler is shown in Figure 6a.

The Primitive Mapping stage serves as the bridge between Python programming abstractions and hardware capabilities. As illustrated in Figure 6b, programmers can express irregular memory operations directly through explicit gather and scatter interfaces. In a standard lowering flow, a feature extraction loop might be translated into eight discrete load operations which incurs significant instruction overhead and memory latency. Through Primitive Mapping, the compiler identifies these patterns and consolidates them into a single hardware gather operation. This transformation effectively leverages the specialized address generation logic within the M-CGRA to achieve superior throughput.

Following this, the compiler performs DFG Generation and Manipulation to optimize the data flow between memory primitives. The resulting DFG reflects the batch level structure required for efficient rendering. The compiler validates batch sizes against hardware limits and shared buffer capacity while rewriting index expressions into batch and inner coordinates. This ensures that the irregular accesses are transformed into a uniform and structured format before they reach the C-CGRA.

After DFG manipulation, the compiler materializes the shared buffer by assigning each batch to a specific bank and base address. It emits the address computation logic that places each encoded element in its correct location within the distributed shared buffer. The resulting layout is bank aware so that C-CGRA tiles can read input batches directly without further reformatting or synchronization.

2) **Co-design with C-CGRA:** For RAICHU IR regions that the algorithm and configuration assign to the C-CGRA, the compiler follows the C-CGRA path illustrated in Figure 6a. This path reshapes arithmetic code into compact vector kernels that align with the parallel structure of each PE. The process

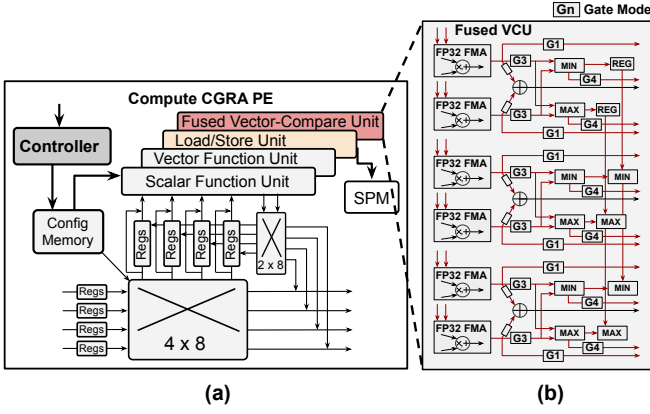


Fig. 7. (a) Compute CGRA PE-level architecture overview. (b) Fused Vector-Compare Unit (FVCU) microarchitecture.

includes pattern matching, operation fusion, dataflow graph construction, and final mapping onto the C-CGRA array.

The key transformation is operation fusion for the FVCU. The compiler identifies arithmetic patterns that naturally map onto the six-lane fused unit. For example, dot-product and matrix-vector computations in Low Rank NeRF are rewritten into partial dot-product and reduction operations. In 3DGS and ray-based shading, paired 3D dot products such as $n \times l$ and $n \times v$ are fused into a single parallel operation, as illustrated in Figure 6c. Ray-tracing operations like axis-aligned box checks are converted into vectorized forms that expose their parallel structure. Single 3D dot products and six-element fused multiply-accumulate chains are likewise collapsed into compact FVCU primitives. Thus, scalar multiply-add chains are replaced by a few fused operations, supported by vector loads matching the FVCU layout.

After fusion, the compiler constructs a dataflow graph that captures the dependencies among vector loads, fused operations, and the remaining scalar steps. This graph is then simplified into a compact kernel block that can be instantiated across all C-CGRA tiles without modification. During mapping, this block is placed and scheduled on the C-CGRA. The shared buffer ensures that each tile receives a coherent batch of data, allowing the compiler to map the same fused graph to every tile while selecting FVCU modes and aligning vector loads with the buffer banks. The regularized batches produced by the M-CGRA eliminate imbalance across tiles and enable the C-CGRA to function as a set of parallel compute units executing an identical fused kernel.

C. Software-Hardware Co-design Pipeline

The proposed co-designed pipeline organizes diverse rendering workloads into a unified execution flow across the M-CGRA and C-CGRA. The M-CGRA maintains multiple batches in flight, advancing each through idle, fetching, and pending states while storing ray state, stage tags, and per-batch data blocks in its local SRAM. In doing so, it absorbs irregular memory accesses, gathers the working set required for each

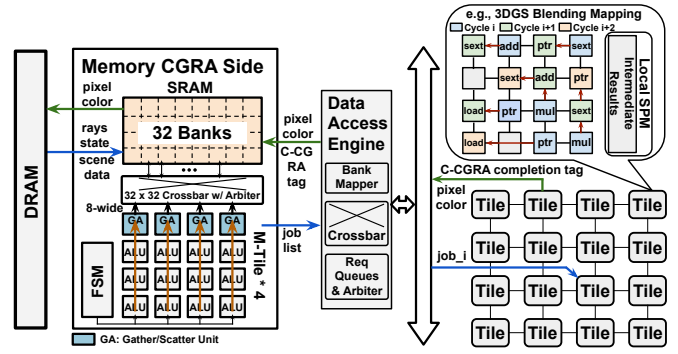


Fig. 8. SW-HW co-designed pipeline on RAICHU.

rendering stage, and performs lightweight preprocessing to produce batch-organized job lists.

These job lists are then forwarded to the DAE, which maps them onto shared-buffer banks and distributes them across the C-CGRA tiles. Each tile executes a fused compute kernel on its assigned jobs and returns intermediate results, completion tags, and pixel colors to the M-CGRA. Together, the DAE and the Memory and Compute CGRAs form a continuous producer-consumer pipeline that exposes locality, balances memory and computation, and sustains high throughput across heterogeneous units.

Figure 8 illustrates the generalized hardware-software co-design pipeline of our architecture. Batches advance through *Idle*, *Fetching*, and *Pending* states under the M-CGRA’s control. During the fetching stage, the M-CGRA gathers required spatial primitives to construct coalesced job lists, effectively masking memory latency. The DAE then dispatches these jobs to the C-CGRA. Here, compiled kernels execute on the PE array, leveraging local SPM for intermediate results. Once complete, C-CGRA tiles return updated data and a completion tag to the M-CGRA, establishing a continuous, high-throughput execution flow.

1) **Hash Grid NeRF**: Hash grid NeRF maps to the M-CGRA by tracking sample positions and partial MLP activations within the ray state. State transitions drive the M-CGRA to gather voxel-corner features across active hash levels, merging them into dense embeddings. The data access engine feeds these embeddings to the C-CGRA, where fused kernels update the MLP stages to maximize on-chip compute utilization.

2) **Low Rank NeRF Pipeline**: Low rank neural radiance fields map to the M-CGRA for coherent embedding generation and task mapping. State transitions enable extensive tile reuse to smooth memory traffic and maximize data locality. The data access engine feeds these tasks to the C-CGRA for high utilization execution.

3) **3DGS Pipeline**: 3D gaussian splatting maps to the M-CGRA to compute conic terms and generate gaussian centric job lists. State transitions facilitate loading each gaussian once while filtering pixels to maximize reuse across neighboring spatial tiles. The data access engine distributes tasks to the

TABLE III

PERFORMANCE COMPARISON OF PROPOSED RAICHU ON FIVE TYPICAL RENDERING PIPELINES FOR SCENES FROM THE NeRF SYNTHETIC DATASET [1] AND LUMIBENCH DATASET [49]: (A) PERFORMANCE SPEEDUP AND (B) ENERGY EFFICIENCY IMPROVEMENT OVER EDGE GPU PLATFORMS, VERSATILE RENDERING ACCELERATORS, AND APPLICATION-SPECIALIZED RENDERING ACCELERATORS. EGP DENOTES EDGE GPU PLATFORM, VRA DENOTES VERSATILE RENDERING ACCELERATOR, AND ASRA DENOTES APPLICATION-SPECIALIZED RENDERING ACCELERATOR.

Platform	Work	M-NeRF	G-NeRF	L-NeRF	3DGS	RT
EGP	Orin NX [59]	33.92×	109.65×	5.67×	16.68×	7.18×
	Nano [60]	135.59×	175.40×	7.43×	19.36×	288.02×
VRA	Uni-Render [39]	4.85×	1.32×	4.36×	1.85×	-
	FlexNeRFer [61]	1.88×	6.09×	0.31×	-	-
ASRA	-	MetaVRain [31]	Instant-3D [62]	RT-NeRF [24]	GSCore [32]	AQB8 [14]
	-	0.44×	7.92×	13.08×	1.01×	3.95×

(a) Performance Speedup

Platform	Work	M-NeRF	G-NeRF	L-NeRF	3DGS	RT
EGP	Orin NX [59]	87.87×	151.60×	10.83×	20.78×	17.58×
	Nano [60]	428.78×	375.70×	19.67×	38.54×	883.01×
VRA	Uni-Render [39]	7.77×	2.12×	6.99×	2.97×	-
	FlexNeRFer [61]	3.81×	12.33×	0.63×	-	-
ASRA	-	MetaVRain [31]	Instant-3D [62]	RT-NeRF [24]	GSCore [32]	AQB8 [14]
	-	0.16×	4.17×	27.18×	0.25×	11.43×

(b) Energy Efficiency Improvement

TABLE IV

RENDERING QUALITY COMPARISON. ENTRIES MARKED WITH AN ASTERISK INDICATE OUR REPRODUCED RESULTS.

Implementation	M-NeRF	G-NeRF	L-NeRF	3DGS
Baseline	31.01	30.85	31.61	33.32
RAICHU	30.81	30.72	31.23	33.48

Note: M-NeRF, G-NeRF, and L-NeRF denote MLP-based NeRF, Hash Grid NeRF, and Low-Rank Decomposition NeRF, respectively.

C-CGRA for fused accumulation and efficient scratchpad memory utilization.

4) *Ray Tracing Pipeline*: Ray tracing maps to the M-CGRA by grouping spatially neighboring rays to share bounding volume hierarchy nodes. State transitions enable coalesced memory requests and node reuse across overlapping ray paths to smooth memory traffic. The data access engine distributes bundled intersection jobs to the C-CGRA for fused kernel evaluation and high throughput execution.

D. Fused Vector Compute Unit

The PE incorporates a compact FVCU that consolidates the short vector patterns pervasive in NeRF, 3DGS and ray tracing. These workloads repeatedly generate 3–6 element Fused Multiply Adds (FMAs), min/max chains, and small dot product groups that are too fine grained to efficiently occupy a C-CGRA tile on their own. The FVCU provides a dedicated datapath to fuse these fragments into a single, high utilization pipeline.

As shown in Figure 7b, each FVCU lane consists of an FP32 FMA followed by a configurable network of min/max units and forwarding switches, coordinated by four gate modes. Gate mode 1 (G1) performs six independent multiply-add operations, directly supporting patterns such as `partial_dot6`. G2 extends this by pairing and summing the six FMA results, enabling the reduce stage commonly used in NeRF and low-rank NeRF. G3 implements the comparison-and-interval updates required for AABB slab tests in ray traversal. G4 performs pairwise compare-and-swap, which is useful for ordering the dual 3D dot products in 3DGS or tightening interval bounds in ray queries. By switching among these modes, the FVCU flexibly captures the dominant vector motifs in rendering pipelines, allowing the compiler to pack heterogeneous small operators into a single, regularized compute path.

By hardening these recurrent motifs, the FVCU keeps PE throughput high even when kernels contain many tiny vector operations. Combined with the M-CGRA’s data reordering, it ensures that the C-CGRA remains compute-bound and fully utilized across diverse rendering workloads.

E. Foveated Rendering Implementation

RAICHU also supports foveated rendering. Specifically, we adopt a unified foveated sampling scheme, consistent with the structure that applies naturally across all rendering methods considered in this work, including NeRF-style volume accumulation, 3D Gaussian Splatting, and ray-tracing-based intersection pipelines. The image plane is divided into 3×3 tiles, and each tile selects a sampling mask based only on its distance from the gaze point. Tiles in the foveal region retain all nine pixels. Tiles in the parafoveal region shade six structured pixels. Perifoveal tiles keep only three diagonal samples, while peripheral tiles are reduced to a single anchor sample.

M-CGRA drives this process by determining the region of each tile, assigning the appropriate sampling mask, and emitting shading tasks only for the marked pixels. Pixels that are not sampled are reconstructed by copying the value of the nearest sampled point inside the same tile, which keeps the reconstruction lightweight while preserving spatial coherence. Because the sampling rule depends only on tile geometry, it forms a shared front end for all rendering pipelines. Whether the downstream stage blends Gaussians, marches rays through a scene, or evaluates hash based encodings, CGRA receives a uniformly thinned set of pixel tasks. This reduces shading cost across all methods while maintaining perceptual fidelity, setting the stage for the co-designed optimizations that follow.

V. EVALUATION

Our evaluation uses PyTorch and CUDA implementations for NeRF and 3DGS, and a Vulkan [12] implementation for ray tracing. We study three representative NeRF variants, KiloNeRF [63], Instant-NGP [6], and TensoRF [7], along with the official 3DGS implementation [2]. To establish baselines and characterize workloads, we profile rendering kernels on two NVIDIA edge devices, Jetson Orin NX [59] and Jetson Nano [60], both used in prior work as VR edge platforms [64], [65]. RAICHU is evaluated using a dedicated simulation

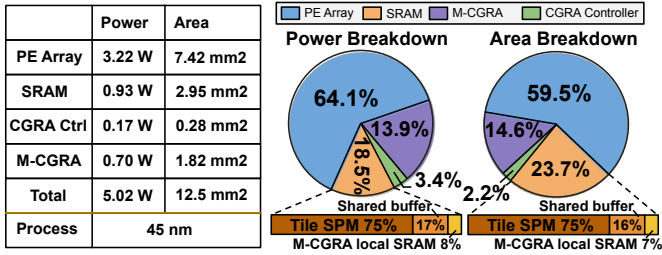


Fig. 9. Power and area breakdown of RAICHU accelerator

framework built on a custom MLIR-based toolchain, with specialized compiler passes for mapping rendering kernels onto the CGRA and enabling detailed latency analysis. A cycle-accurate simulator is used to measure end-to-end pipeline performance. The simulator is a custom accelerator level cycle simulator driven by compiler-generated CGRA mappings. It executes mapped kernels cycle by cycle while accounting for data dependencies, resource availability, communication latency, SRAM bandwidth and bank constraints. We validate functional outputs against the original PyTorch, CUDA, Vulkan implementations and set timing parameters and resource constraints according to the RTL microarchitecture. The simulator models off-chip memory with the assumption of a 102.4 GB/s DRAM bandwidth, which is the same as the typical bandwidth of the LPDDR5 memory subsystem used in the baseline Jetson Orin NX device [59].

We also implement the DAE and CGRA in RTL, with synthesizable CGRA RTL generated using VecPAC [66]. We verify the RTL functionality using module-level and integration-level testbenches that cover DAE request scheduling, arbitration, shared-SRAM access, CGRA PE/FVCU execution, routing, memory operations, and end-to-end data movement. The design is synthesized in Synopsys Design Compiler targeting a 45 nm library to report area, power, and maximum frequency. On-chip SRAM area and energy are modeled with CACTI-7.0 [67], and DeepScaleTool [68] is used to normalize power across process nodes for fair comparison.

A. Algorithm Evaluation

In this section, we assess the effect of our proposed algorithmic optimizations on rendering quality. Specifically, we integrate the pipeline reordering techniques described in Section III-C into all the rendering pipelines. We also apply FP8 quantization to the exponential function in the 3DGS blending stage and to the MLPs in the NeRF pipeline. The effect of the reordered pipeline presented in Section III-C on image quality is quantitatively evaluated using the NeRF Synthetic Dataset, with Peak Signal-to-Noise Ratio (PSNR) as the metric. As summarized in Table IV, the results show that across all evaluated scenes and methods, the PSNR degradation remains consistently below 1 dB relative to baselines. Since a difference of less than 1 dB is typically imperceptible in visual quality and falls well below established human per-

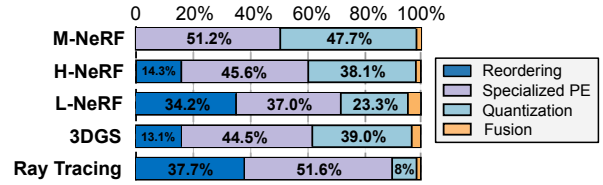


Fig. 10. Contribution of each component to the speedup of different rendering methods. M-NeRF, H-NeRF, and L-NeRF denote MLP-based, Hash Grid, and Low-Rank NeRF.

ceptual thresholds [69], this confirms that RAICHU algorithm preserves rendered image fidelity with negligible impact.

B. Hardware Evaluation

1) **Power & Area Breakdown.**: We synthesize the RAICHU accelerator using Synopsys Design Compiler with a FreePDK 45nm library [70] at a target frequency of 1 GHz to obtain detailed power and area estimates. For energy, we model the off-chip DRAM access energy as 8 pJ/bit [71] multiplied by the per-method DRAM traffic from the simulator. On chip SRAM area and energy are characterized separately using CACTI 7.0. As shown in Figure 9, the accelerator consumes 5.02 W in total. The C-CGRA array is the primary contributor, accounting for 64.1% of overall power, followed by SRAM at 18.5% and the M-CGRA plus DAE at 13.9%. The total chip area is 12.5 mm², with the C-CGRA array occupying 59.5%, SRAM 23.7%, and the M-CGRA plus DAE 14.6%.

We compare RAICHU with commercial edge devices [59], [60], versatile rendering accelerators [39], [61], and application-specific accelerators [14], [24], [31], [32], [62], evaluating latency and energy efficiency across different rendering methods, as summarized in Table III. NeRF and 3DGS workloads are profiled using the NeRF Synthetic dataset [1], while ray tracing is evaluated with LumiBench [49].

2) **Comparison over Edge GPU**: We use Jetson Orin NX and Jetson Nano as representative edge platforms in our evaluation. For energy comparison, RAICHU power refers to the synthesized accelerator subsystem, while Jetson energy is measured as idle-subtracted module-level dynamic energy [72], [73] using the onboard power monitors, since a GPU-only power rail is not exposed on these platforms [74]. To match the module level Jetson measurements, which already include off-chip DRAM power, RAICHU energy in this section also includes off-chip DRAM access energy, as described in Section V-B1. Using NeRF Synthetic dataset [1] for neural radiance fields and 3D gaussian splatting alongside LumiBench [49] for ray tracing, RAICHU achieves 5.67 $\times$ to 288.02 $\times$ speedup and 10.83 $\times$ to 883.01 $\times$ energy efficiency gains over both edge GPU baselines across all five methods. For ray tracing workloads, the Orin includes dedicated RT Cores, whereas the Nano lacks specialized ray tracing hardware and must execute highly divergent kernels on CUDA Cores. As a result, RAICHU achieves a much larger speedup over the Nano than over the Orin on ray tracing tasks.

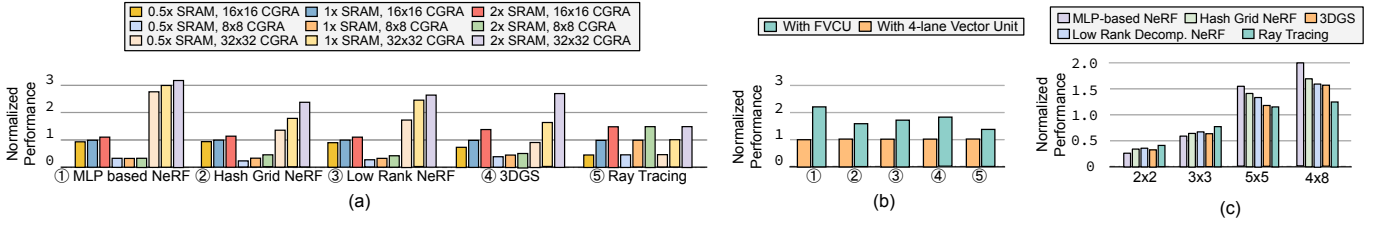


Fig. 11. (a) Normalized performance of five rendering workloads: ① MLP-based NeRF, ② Hash-Grid NeRF, ③ Low-Rank NeRF, ④ 3DGS, and ⑤ Ray Tracing, when sweeping M-CGRA side SRAM/bandwidth ($0.5\times$, $1\times$, $2\times$) and CGRA array size (8×8 , 16×16 , 32×32). Performance is normalized to the baseline configuration with $1\times$ M-CGRA SRAM and a 16×16 CGRA. (b) Performance with the proposed FVCU versus a design where the FVCU is replaced by a conventional 4-lane vector unit. Bars are normalized to the 4-lane vector unit design (larger is better). (c) Speedup of different rendering methods while scaling C-CGRA.

3) Comparison with Other Versatile Rendering Accelerators: We select two existing versatile rendering accelerators including Uni Render [39] and FlexNeRFer [61] to compare with the proposed RAICHU. As shown in Table III, compared with UniRender, RAICHU achieves $1.32\times$ – $4.85\times$ speedup and $2.12\times$ – $7.77\times$ higher energy efficiency, while providing additional acceleration support for the ray tracing pipeline. Compared with FlexNeRFer, RAICHU delivers $0.31\times$ – $6.09\times$ speedup and $0.63\times$ – $12.33\times$ higher energy efficiency, while additionally supporting both 3DGS and ray tracing. Overall, RAICHU provides higher performance and energy efficiency than existing versatile designs while supporting a broader set of rendering pipelines. This advantage comes from its heterogeneous CGRA architecture and specialized compiler, jointly exploiting specific computation patterns through SW-HW co-design.

4) Comparison with Application Specific Rendering Accelerators: As shown in Table III, RAICHU achieves $0.44\times$ – $13.08\times$ speedup and $0.16\times$ – $27.18\times$ higher energy efficiency. RAICHU shows lower performance than MetaVRain on MLP-based NeRF, partly because MetaVRain is specifically optimized for this pipeline. In addition, RAICHU is mainly designed to accelerate pipelines that contain a memory-bound stage followed by a compute-bound stage. However, as discussed in Section III-A, the latency of MLP-based NeRF is dominated almost entirely by MLP computation, which limits RAICHU’s ability to fully exploit its architectural advantages. For the other specialized accelerators, RAICHU achieves better results on nearly all pipelines, demonstrating strong competitiveness across Hash Grid NeRF, Low Rank NeRF, 3DGS, and ray tracing.

5) Speedup Mechanisms: RAICHU’s speedups come from matching each workload bottleneck to the M-CGRA/C-CGRA execution path in Section IV-C. MLP-based NeRF is compute-dominated, so its gain mainly comes from C-CGRA/FVCU fused arithmetic; this also explains why MetaVRain [31], which is fully specialized for MLP-based NeRF, can outperform RAICHU on this single pipeline. For Hash Grid NeRF and Low-Rank NeRF, the mechanisms follow the mappings in Sections IV-C1 and IV-C2: the M-CGRA batches shared hash vertices or plane/line factors across nearby samples before the C-CGRA performs interpolation and MLP

execution, thereby reducing redundant feature traffic. For 3DGS, the mapping in Section IV-C3 leverages its natural Gaussian-centric structure: the M-CGRA loads each Gaussian once per job batch, filters its affected pixels or tiles, and emits compact jobs to C-CGRA, reducing repeated Gaussian parameter movement through the pipeline. However, these gains rely on spatial locality; lower locality reduces reusable data sharing and therefore limits the performance gain.

For ray tracing, the Section IV-C4 mapping exploits rays locality: spatially neighboring rays are usually adjacent in the ray order and thus likely fall into the same batch. The M-CGRA coalesces shared BVH node and triangle fetches along overlapping paths without an extra grouping stage, while the C-CGRA evaluates bundled intersection jobs from shared buffers. However, at deeper BVH levels, ray paths diverge and node sharing drops, making performance more M-CGRA bandwidth-bound.

Overall, these mechanisms reflect a deliberate tradeoff in RAICHU. The M-CGRA/C-CGRA split and FVCU specialization, as discussed in Section III-D, reduce generality and make RAICHU less suitable for arbitrary workloads, while improving efficiency for rendering pipelines with shareable irregular accesses and dense arithmetic.

C. Ablation Study

1) M-CGRA SRAM Capacity vs. C-CGRA Array Size:

We sweep M-CGRA buffering and bandwidth together with C-CGRA array size, as shown in Figure 11a. M-CGRA resources are scaled to $0.5\times$, $1\times$, and $2\times$ the baseline, while the C-CGRA array ranges from 8×8 (64 PEs) to 32×32 (1024 PEs). Performance is normalized to the configuration with $1\times$ SRAM and a 16×16 C-CGRA. Sensitivity to hardware scaling varies substantially across workloads. MLP-based NeRF is purely compute-bound and scales linearly with C-CGRA size, with little dependence on M-CGRA capacity. Hash Grid NeRF and Low Rank NeRF are also mainly C-CGRA bound, but show modest gains from larger M-CGRA resources at bigger array sizes due to improved feature sharing. In contrast, 3DGS requires balanced scaling of both units: while small configurations are compute-limited, scaling the C-CGRA to 32×32 yields significant benefits only when M-CGRA resources are also doubled to relieve memory pressure. Ray

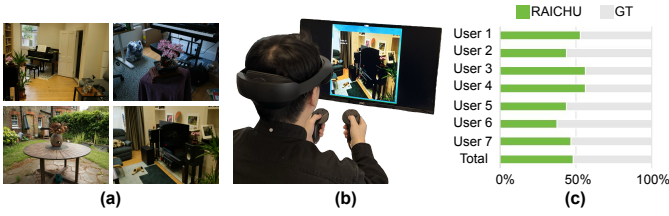


Fig. 12. (a) Four test images rendered by RAICHU 3DGS. (b) Participants taking part in the study. (c) Selection results.

tracing, by contrast, is strictly memory-bound. Its performance scales almost linearly with M-CGRA capacity, while larger C-CGRA arrays provide little benefit. These results show that area allocation between M-CGRA and C-CGRA should be guided by the dominant rendering characteristics of the target workload.

2) **PE Array Scaling within C-CGRA Tiles:** We further study the effect of scaling the PE array size within each C-CGRA tile while fixing the total number of tiles at 4×4 . Using the 4×4 PE array as the baseline, we evaluate 2×2 , 3×3 , 5×5 , and 4×8 configurations across the five rendering pipelines, as shown in Figure 11c.

For MLP-based NeRF, performance scales almost proportionally with PE array size, confirming that this workload is strongly compute-bound and dominated by MLP execution. In contrast, the other four pipelines do not show proportional gains as the PE array grows, suggesting that the baseline already lies near an effective design point for these workloads. Increasing C-CGRA compute resources alone, without matching improvements in memory access, disrupts the balance between data orchestration and computation. Ray tracing benefits the least from this scaling. As a representative memory-bound workload, it sees little improvement from additional compute units once memory bandwidth becomes the main bottleneck. These results highlight the need to balance M-CGRA and C-CGRA resources to avoid underutilizing larger compute arrays.

3) **FVCU versus a Four Lane Vector Unit:** We evaluate the benefits of the specialized FVCU by comparing it with a standard four-lane SIMD vector unit while keeping the M-CGRA and C-CGRA configurations at their baseline values. As shown in Figure 11b, the specialized FVCU is important for compute-intensive rendering tasks. MLP-based NeRF achieves a $2.2\times$ speedup because its matrix-vector layers map directly to the FVCU’s specialized partial dot modes. Hash Grid NeRF and Low Rank NeRF similarly gain $1.6\times$ – $1.7\times$ from more efficient dot-product support in their encoding and MLP layers. 3DGS improves by $1.8\times$, as Gaussian blending kernels benefit from specialized dot2x3 and fused multiply-add modes. In contrast, ray tracing is less sensitive to FVCU scaling because it is mainly limited by M-CGRA traffic. Replacing the FVCU with a four-lane unit reduces ray tracing performance by only $1.3\times$, mainly due to slower AABB tests and comparison operations. Overall, these results justify our design choice: NeRF and 3DGS benefit most from specialized C-CGRA compute, while ray tracing depends more on M-CGRA resources. This

TABLE V
THROUGHPUT OF RAICHU UNDER FOVEATED SETTING.

Pipeline	MLP	Low-rank NeRF	Hash Grid NeRF	3DGS	Ray Tracing
Ref. FPS	[1] 359	[7] 134	[6] 428	[2] 313	[49] 195

supports the inclusion of a modestly larger but more efficient FVCU in each PE.

4) **Speedup Breakdown:** We evaluate the individual contributions of Pipeline Reordering, Specialized PE, Quantization, and Operation Fusion. As shown in Figure 10, we measure each contribution using logarithmic speedup values. The heterogeneous M-CGRA/C-CGRA architecture together with Pipeline Reordering accounts for 13.1%–37.7% of the total speedup, confirming that the data orchestration optimizations in Section III-C effectively reduce memory bottlenecks. Although MLP-based NeRF does not use reordering because it is compute-dominated, the other optimizations still provide substantial gains. Specialized PE modules contribute the most across most workloads. By accelerating common rendering operations with dedicated PE support and compiler-level fusion, RAICHU achieves high execution efficiency. Quantization, from FP8 to FP16, further contributes 8%–47.7% through MLP computation, blending, and intersection tests. The contribution of general Operation Fusion is smaller because much of the rendering-specific fusion is already captured in the Specialized PE category. Overall, these results show that the combination of reconfigurable memory orchestration and specialized compute support is the main source of RAICHU’s performance gains.

D. User Study

To evaluate our RAICHU-based foveated 3DGS, we conduct a user study to determine whether foveated rendering introduces perceivable visual artifacts. The goal is to assess visual quality under gaze-contingent viewing and verify that RAICHU maintains high-quality VR experiences. We recruit seven participants, each seated and viewing stimuli through a Meta Quest Pro HMD [75], using the controllers to switch between images across trials. The stimuli are generated by rendering multiple scenes with 3DGS. We first select four representative viewpoints from four scenes (Figure 12a). For each view, we use the full-resolution 3DGS image as the ground-truth (GT) reference and render a corresponding foveated image with RAICHU using a 3×3 upsampling layout centered on the screen, emulating a central gaze position. Each viewpoint therefore yields a GT/RAICHU image pair.

Participants then complete a two-interval forced-choice (2IFC) task [76]. In each trial, two images of the same scene and viewpoint were shown (t1 and t2), representing the ground-truth and RAICHU outputs in randomized order. Participants freely toggle between the two using the controllers and were instructed to keep their gaze near a central fixation point while selecting the image with higher perceived quality.

TABLE VI
EXPOSED FRAME-LEVEL RECONFIGURATION LATENCY.

Workload	Ideal	No Pref.	Prefetch	Fusion
RT + NeRF	2.664 ms	169.34 μ s 6.355%	3.22 μ s 0.121%	0.03 μ s 0.00113%
RT + 3DGS	3.430 ms	4.24 μ s 0.124%	1.31 μ s 0.038%	0.03 μ s 0.00087%
RT + 3DGS + NeRF	3.038 ms	171.92 μ s 5.659%	3.23 μ s 0.106%	0.04 μ s 0.00132%

Each participant complete 32 trials, covering four image pairs with two presentation orders and four repetitions, providing a compact yet diverse set of comparisons across scenes and randomized conditions. The aggregated results are shown in Figure 12c. Across all trials, the RAICHU foveated images were chosen in $48.2\% \pm 7.2\%$ of cases, essentially matching chance. This near-equal preference indicates that participants could not reliably distinguish RAICHU from GT, suggesting that the foveated output preserves visual quality and is suitable for practical VR deployment.

E. Throughput under Foveated Setting

With the combined hardware-software optimizations described above, together with our foveated rendering strategy, RAICHU’s throughput is summarized in Table V. Most evaluated pipelines exceed the 120 FPS target for high-end immersive experiences. Specifically, MLP-based NeRF reaches 359 FPS, hash-grid NeRF achieves 428 FPS, 3DGS reaches 313 FPS, and ray tracing delivers 195 FPS, while low-rank NeRF reaches 134 FPS. Overall, these results show that RAICHU delivers real-time performance across a diverse range of modern rendering methods.

F. Reconfiguration Analysis

1) **Reconfiguration overhead.**: RAICHU supports different rendering stages through compile time generated context programs. We distinguish intra-kernel context sequencing from inter-kernel configuration loading. During kernel execution, each PE selects a context already resident in its local context bank; this cost is included in the reported kernel latency. Inter-kernel configuration loading is modeled separately and can be overlapped with execution through double-buffered context banks and prefetching.

Table VI reports the exposed frame level reconfiguration latency for hybrid rendering workloads. Without prefetching, stage level switching can add up to 6.355% latency. With double-buffered prefetching, the exposed overhead is reduced to 0.038%–0.121%. When stages in the same rendering method are fused into one context program per CGRA, the exposed overhead further drops to 0.00087%–0.00132%, corresponding to only 30–40 cycles per frame.

2) **Configurability and support for new variants.**: RAICHU uses the same hardware template for the evaluated NeRF, 3DGS, and ray tracing workloads. Its configurability comes from compile-time dataflow generation and CGRA

TABLE VII
SUPPORT PATHS FOR RENDERING ALGORITHM VARIANTS.

Variant Type	Existing Compiler	Existing Hardware
Evaluated kernels	✓	✓
Supported micro operators	✓	✓
New dataflow/access pattern	✗	✓
New specialized primitive	✗	✗

reconfiguration, where the compiler configures the M-CGRA, C-CGRA, and DAE.

RAICHU-supported micro operations cover common rendering computations such as transformation, interpolation, filtering, comparison, accumulation, and blending. Therefore, many new rendering variants can be decomposed into these micro operations and supported by recompilation. As summarized in Table VII, compiler extensions are needed when a variant requires new dataflow/access pattern, while hardware extensions are needed only for new specialized primitives that are inefficient on existing RAICHU units.

VI. CONCLUSION

We introduced RAICHU, a unified CGRA-based accelerator for NeRF, 3DGS, and ray tracing. Through hardware–software co-design, RAICHU transforms irregular rendering workloads into efficient compute batches, achieving large gains in performance and energy efficiency over edge GPUs while preserving visual quality. These results show that a programmable spatial architecture can effectively support diverse real-time VR rendering pipelines.

APPENDIX

A. Abstract

This artifact provides the compiler, cycle-level simulator, and synthesizable RTL used to evaluate RAICHU. A unified CPU-only Docker environment builds the required LLVM/MLIR toolchain, maps representative rendering kernels, exercises the versioned compiler, simulator, and RTL interfaces, and runs the simulator and RTL functional tests. The artifact also provides one-command experiments for reproducing the five rendering-method speedups in Table 3(a) and the 2,560-cycle initial configuration-transfer result in Section 5.2. All inputs, paper baselines, expected results, and machine-readable hardware and workload descriptions needed by these experiments are included in the archive.

B. Artifact check-list (meta-information)

- **Algorithm:** Trace-driven simulation and dataflow mapping; double-buffered CGRA configuration transfer.
- **Program:** RAICHU simulator, MLIR compiler, PyMTL RTL, and Bash/Python scripts.
- **Compilation:** CMake/Ninja LLVM/MLIR build; mlir-neura-opt; PyMTL translation; Verilator lint and simulation.
- **Transformations:** Rendering-kernel lowering, scheduling, PE mapping, and versioned compiler-trace export.

- **Binary:** The compiler binary is built inside the supplied Docker image; no prebuilt proprietary binary is required.
- **Model:** Deterministic RAICHU cycle-level execution, memory, interconnect, and reconfiguration models; RTL-derived hardware profile.
- **Data set:** Packaged 35-kernel mapper export, dataset-average workload inputs and paper baselines.
- **Run-time environment:** CPU-only Ubuntu 22.04 Docker container.
- **Hardware:** x86-64 host; 16 CPU cores, 32 GB RAM, and 60 GB free disk recommended. No GPU, FPGA, Jetson device, or proprietary EDA tool is required for the claimed workflow.
- **Run-time state:** No special run-time state or network access is required after the Docker image has been built.
- **Execution:** Automated shell scripts with nonzero exit status on validation or result-checking failure.
- **Metrics:** Cycles, latency, FPS, speedup, configuration-transfer time, and functional test status.
- **Output:** Markdown/CSV/JSON result summaries, event manifests, mapper results, generated RTL, and Verilator reports.
- **Experiments:** Functional integration, Table 3(a) reproduction, Section 5.2 reconfiguration reproduction, and optional component tests.
- **How much disk space required (approximately)?:** 60 GB, primarily for the LLVM/MLIR build and Docker layers.
- **How much time is needed to prepare workflow (approximately)?:** Several hours for the first Docker build, depending on CPU and network speed.
- **How much time is needed to complete experiments (approximately)?:** Up to approximately one hour on a 16-core host; the exact time depends on compiler mapping and RTL translation performance.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** MIT for the compiler; BSD 3-Clause for the simulator and RTL; retained upstream notices apply to vendored RTL sources.
- **Data licenses (if publicly available)?:** No external raw data set is redistributed; packaged experiment metadata follows the component-local license and provenance notices.
- **Workflow automation framework used?:** Docker and Bash.
- **Archived (provide DOI)?:** DOI 10.5281/zenodo.21537289.

C. Description

1) *How to access:* Download RAICHU_AE-v1.0.0-ae.tar.gz and its .sha256 sidecar from DOI 10.5281/zenodo.21537289. The archive contains a top-level reviewer guide, a package-wide AE_MANIFEST.json, and independent guides under simulator/, compiler/raichu_ae/, and rtl/docs/.

2) *Hardware dependencies:* The claimed workflow is CPU-only. We recommend an x86-64 Linux host with at least 16 CPU cores, 32 GB RAM, and 60 GB free disk space. Docker may also run through a compatible Linux-container environment on another host OS.

3) *Software dependencies:* The host needs Docker Engine with Linux-container support and Bash. The provided Dockerfile installs Ubuntu 22.04, Python 3.11, CMake 3.28.3, LLVM/MLIR/Clang commit 6146a88f6049, PyMTL3 3.1.13, pytest 7.4.4, and Verilator 5.026. The complete commit hash is pinned in the Dockerfile. Internet access is needed for the first image build. No proprietary software or license is required for the claimed experiments.

4) *Data sets:* No external data download is required. The archive includes the compiler inputs and mapper results, simulator scenarios, paper baselines, expected results, workload-average inputs, and RTL hardware profile.

5) *Models:* The simulator consumes the versioned raichu.compiler-trace.v1 and raichu.hardware-profile.v1 formats and emits a checksummed run manifest. The RTL profile describes the released 256-PE design. Its topology and resource counts come from the RTL, while target frequency and selected latency/bandwidth parameters remain explicit model inputs.

D. Installation

On the host, verify and unpack the archive:

```
sha256sum -c \
  RAICHU_AE-v1.0.0-ae.tar.gz.sha256
tar -xzf RAICHU_AE-v1.0.0-ae.tar.gz
cd RAICHU_AE
./docker/build.sh
./docker/run.sh
```

The first build compiles LLVM/MLIR, Verilator, and the RAICHU compiler and can take several hours. The final command opens a shell in the evaluation container and maps outputs/ to the host directory ae_outputs/.

E. Experiment workflow

Inside the container, run the complete Functional workflow:

```
./scripts/run_functional.sh
```

This validates the archive manifest and environment, runs 75 simulator tests and golden simulations, maps the representative H-NeRF kernels, checks both cross-project interfaces, runs the Table 3(a) and reconfiguration experiments, and executes the RTL smoke, Verilator-import, generation, and lint checks.

The two paper-result experiments can also be run independently:

```
./scripts/run_table3a.sh
cat outputs/table3a/table3a_results.md

./scripts/run_reconfiguration.sh
cat outputs/reconfiguration/*results.md
```

F. Evaluation and expected results

The artifact evaluation produces results that closely match the values reported in the paper. Table 3(a) is evaluated for all five rendering methods using the compiler-generated mapping statistics and the integrated simulator. The resulting speedups and their relative differences from the paper are:

Method	Paper	Artifact	Difference
MLP-based NeRF	33.9×	34.158×	+0.76%
HashGrid NeRF	109.7×	109.690×	-0.01%
Low-Rank NeRF	5.7×	5.700×	0.00%
3DGS	16.7×	16.700×	0.00%
Ray Tracing	7.2×	7.187×	-0.18%

The evaluation therefore shows strong numerical agreement: the maximum relative difference is only 0.76%, and the mean absolute relative difference is 0.19%. All five results pass the artifact's numerical checks. The small apparent differences are consistent with the limited precision of the one-decimal Orin FPS and speedup values reported in Figure 3(b) and Table 3(a); the checker propagates the rounding intervals of both operands.

The reconfiguration experiment measures 2,560 transfer cycles, equivalent to $2.56\mu\text{s}$ at 1GHz. This result exactly matches the value reported in Section 5.2. After all compiler, simulator, RTL, and cross-project checks pass, `run_functional.sh` exits with status zero and reports:

```
[functional] compiler, simulator, RTL,
and cross-project checks passed
```

G. Experiment customization

Reviewers may run each component independently:

- Simulator: `simulator/scripts/run_ae.sh`
- Compiler: `compiler/raichu_ae/run_ae.sh`
- RTL: `rtl/scripts/run_smoke.sh`

The compiler accepts `mnerf`, `hnerf`, `lnerf`, `3dgs`, or `rt`; running all methods should end with `PASS=35 FAIL=0`. The longer `rtl/scripts/run_end_to_end.sh` regenerates and lints the full 256-PE RTL. Output directories can be redirected with the environment variables documented in the top-level reviewer guide.

REFERENCES

- [1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, p. 99–106, Dec. 2021. [Online]. Available: <https://doi.org/10.1145/3503250>
- [2] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, Jul. 2023. [Online]. Available: <https://doi.org/10.1145/3592433>
- [3] R. Albert, A. Patney, D. Luebke, and J. Kim, "Latency requirements for foveated rendering in virtual reality," *ACM Transactions on Applied Perception (TAP)*, vol. 14, no. 4, pp. 1–13, 2017.
- [4] A. Tewari, J. Thies, B. Mildenhall, P. Srinivasan, E. Tretschk, Y. Wang, C. Lassner, V. Sitzmann, R. Martin-Brualla, S. Lombardi, T. Simon, C. Theobalt, M. Niessner, J. T. Barron, G. Wetzstein, M. Zollhofer, and V. Golyanik, "Advances in neural rendering," 2022. [Online]. Available: <https://arxiv.org/abs/2111.05849>
- [5] A. Tewari, O. Fried, J. Thies, V. Sitzmann, S. Lombardi, K. Sunkavalli, R. Martin-Brualla, T. Simon, J. Saragih, M. Nießner, R. Pandey, S. Fanello, G. Wetzstein, J.-Y. Zhu, C. Theobalt, M. Agrawala, E. Shechtman, D. B. Goldman, and M. Zollhöfer, "State of the art on neural rendering," 2020. [Online]. Available: <https://arxiv.org/abs/2004.03805>
- [6] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, Jul. 2022. [Online]. Available: <https://doi.org/10.1145/3528223.3530127>
- [7] A. Chen, Z. Xu, A. Geiger, J. Yu, and H. Su, "Tensorf: Tensorial radiance fields," in *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXXII*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 333–350. [Online]. Available: https://doi.org/10.1007/978-3-031-19824-3_20
- [8] H. Turki, D. Ramanan, and M. Satyanarayanan, "Mega-nerf: Scalable construction of large-scale nerfs for virtual fly-throughs," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12912–12921.
- [9] T. Xie, Z. Zong, Y. Qiu, X. Li, Y. Feng, Y. Yang, and C. Jiang, "PhysGaussian: Physics-Integrated 3D Gaussians for Generative Dynamics," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA: IEEE Computer Society, Jun. 2024, pp. 4389–4398. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CVPR52733.2024.00420>
- [10] Z. Shao, Z. Wang, Z. Li, D. Wang, X. Lin, Y. Zhang, M. Fan, and Z. Wang, "Splattingavatar: Realistic real-time human avatars with mesh-embedded gaussian splatting," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 1606–1616.
- [11] B. M. BIJU, S. P. M., and S. K. M., "Nvidia ray tracing: Revolutionizing real-time computer graphics," *TechRxiv*, vol. 2025, no. 0410, 2025. [Online]. Available: <https://www.techrxiv.org/doi/abs/10.36227/techrxiv.174431738.81337106/v1>
- [12] M. Saed, Y. H. Chou, L. Liu, T. Nowicki, and T. M. Aamodt, "Vulkan-sim: A gpu architecture simulator for ray tracing," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 263–281.
- [13] Y. S. Tozlu and H. Zhou, "Cooprt: Accelerating bvh traversal for ray tracing via cooperative threads," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 166–179. [Online]. Available: <https://doi.org/10.1145/3695053.3731118>
- [14] Y.-C. Huang, C.-P. Yang, and T. T. Yeh, "Aqb8: Energy-efficient ray tracing accelerator through multi-level quantization," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 374–387. [Online]. Available: <https://doi.org/10.1145/3695053.3731104>
- [15] P. Lancheres and M. Hafeed, "The mipi c-phy standard: A generalized multiconductor signaling scheme," *IEEE Solid-State Circuits Magazine*, vol. 11, no. 2, pp. 69–77, 2019.
- [16] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Comput. Surv.*, vol. 52, no. 6, Oct. 2019. [Online]. Available: <https://doi.org/10.1145/3357375>
- [17] H. Park, K. Fan, S. Mahlke, T. Oh, H. Kim, and H.-s. Kim, "Edge-centric modulo scheduling for coarse-grained reconfigurable architectures," in *2008 International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2008, pp. 166–176.
- [18] Z. Zhao, W. Sheng, Q. Wang, W. Yin, P. Ye, J. Li, and Z. Mao, "Towards higher performance and robust compilation for cgra modulo scheduling," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 9, pp. 2201–2219, 2020.
- [19] C. Ebeling, D. C. Cronquist, and P. Franklin, "Rapid - reconfigurable pipelined datapath," in *Proceedings of the 6th International Workshop on Field-Programmable Logic, Smart Applications, New Paradigms and Compilers*, ser. FPL '96. Berlin, Heidelberg: Springer-Verlag, 1996, p. 126–135.
- [20] H. Singh, M.-H. Lee, G. Lu, F. Kurdahi, N. Bagherzadeh, and E. Filho, "Morphosys: a reconfigurable architecture for multimedia applications," in *Proceedings. XI Brazilian Symposium on Integrated Circuit Design (Cat. No.98EX216)*, 1998, pp. 134–139.
- [21] D. Shin, J. Lee, J. Lee, J. Lee, and H.-J. Yoo, "Dnpu: An energy-efficient deep-learning processor with heterogeneous multi-core architecture," *IEEE Micro*, vol. 38, no. 5, p. 85–93, Sep. 2018. [Online]. Available: <https://doi.org/10.1109/MM.2018.053631145>
- [22] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 389–402.
- [23] Y. Zhang, J. Liang, J. Peng, J. Xu, and W. Zhang, "Spnerf: Memory efficient sparse volumetric neural rendering accelerator for edge devices," 2025. [Online]. Available: <https://arxiv.org/abs/2505.08191>
- [24] C. Li, S. Li, Y. Zhao, W. Zhu, and Y. Lin, "Rt-nerf: Real-time on-device neural radiance fields towards immersive ar/vr rendering," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3508352.3549380>
- [25] J. Lee, K. Choi, J. Lee, S. Lee, J. Whangbo, and J. Sim, "Neurex: A case for neural rendering acceleration," in *Proceedings of the 50th*

- Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589056>
- [26] Y. Feng, Z. Liu, J. Leng, M. Guo, and Y. Zhu, "Cicero: Addressing algorithmic and architectural bottlenecks in neural rendering by radiance warping and memory optimizations," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1293–1308.
- [27] J. Ryu, H. Kwon, W. Park, Z. Li, B. Kwon, D. Han, D. Im, S. Kim, H. Joo, M. Kim, and H.-J. Yoo, "Neugpu: An energy-efficient neural graphics processing unit for instant modeling and real-time rendering on mobile devices," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 1, pp. 99–111, 2025.
- [28] Y. Chen, Z. Li, D. Lyu, Y. Xu, and G. He, "Neural rendering acceleration with deferred neural decoding and voxel-centric data flow," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 7, pp. 2725–2737, 2025.
- [29] L. Wu, H. Zhu, J. Zheng, M. Li, Y. Cheng, Q. Liu, X. Zeng, and C. Chen, "Hi-nerf: A multicore nerf accelerator with hierarchical empty space skipping for edge 3-d rendering," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 12, pp. 2315–2326, 2024.
- [30] H. Wan, L. Ma, A. Li, P. Zhou, J. Yu, and X. Lou, "Zerotetris: A spacial feature similarity-based sparse mlp engine for neural volume rendering," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3655684>
- [31] D. Han, J. Ryu, S. Kim, S. Kim, J. Park, and H.-J. Yoo, "Metavrain: A mobile neural 3-d rendering processor with bundle-frame-familiarity-based nerf acceleration and hybrid dnn computing," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 1, pp. 65–78, 2024.
- [32] J. Lee, S. Lee, J. Lee, J. Park, and J. Sim, "Gscore: Efficient radiance field rendering via architectural support for 3d gaussian splatting," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 497–511. [Online]. Available: <https://doi.org/10.1145/3620666.3651385>
- [33] J. Jo and J. Park, "Ps-gs: Group-wise parallel rendering with stage-wise complexity reductions for real-time 3d gaussian splatting," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025, pp. 1–7.
- [34] Z. Ye, Y. Fu, J. Zhang, L. Li, Y. Zhang, S. Li, C. Wan, C. Wan, C. Li, S. Prathipati, and Y. C. Lin, "Gaussian blending unit: An edge gpu plug-in for real-time gaussian-based rendering in ar/vr," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 353–365.
- [35] Y. Wang, Y. Li, J. Chen, J. Yu, and K. Wang, "Famers: An fpga accelerator for memory-efficient edge-rendered 3d gaussian splatting," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025, pp. 1–7.
- [36] Y. H. Chou and T. M. Aamodt, "Treelet accelerated ray tracing on gpus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1334–1347. [Online]. Available: <https://doi.org/10.1145/3676641.3716279>
- [37] K. Vaidyanathan, S. Woop, and C. Benthin, "Wide bvh traversal with a short stack," in *Proceedings of the Conference on High-Performance Graphics*, ser. HPG '19. Goslar, DEU: Eurographics Association, 2022, p. 15–19. [Online]. Available: <https://doi.org/10.2312/hpg.20191190>
- [38] L. Liu, W. Chang, F. Demoullin, Y. H. Chou, M. Saed, D. Pankratz, T. Nowicki, and T. M. Aamodt, "Intersection prediction for accelerated gpu ray tracing," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 709–723. [Online]. Available: <https://doi.org/10.1145/3466752.3480097>
- [39] C. Li, S. Li, L. Jiang, J. Zhang, and Y. C. Lin, "Uni-render: A unified accelerator for real-time rendering across diverse neural renderers," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 246–260.
- [40] Y. Feng, W. Lin, Z. Liu, J. Leng, M. Guo, H. Zhao, X. Hou, J. Zhao, and Y. Zhu, "Potamoi: Accelerating neural rendering via a unified streaming architecture," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 4, Nov. 2024. [Online]. Available: <https://doi.org/10.1145/3689340>
- [41] C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, and J. D. Tardos, "Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam," *IEEE Transactions on Robotics*, vol. 37, no. 6, p. 1874–1890, Dec. 2021. [Online]. Available: <http://dx.doi.org/10.1109/TRO.2021.3075644>
- [42] W. Liu, B. Duinkharjav, Q. Sun, and S. Q. Zhang, "Fovealnet: Advancing ai-driven gaze tracking solutions for efficient foveated rendering in virtual reality," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 5, p. 3183–3193, May 2025. [Online]. Available: <https://doi.org/10.1109/TVCG.2025.3549577>
- [43] J. M. P. van Waveren, "The asynchronous time warp for virtual reality on consumer hardware," in *Proceedings of the 22nd ACM Conference on Virtual Reality Software and Technology*, ser. VRST '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 37–46. [Online]. Available: <https://doi.org/10.1145/2993369.2993375>
- [44] S. Li, B. Keller, Y. C. Lin, and B. Khailany, "Gaurast: Enhancing gpu triangle rasterizers to accelerate 3d gaussian splatting," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7.
- [45] L. Li, J. Qin, J. Peng, Z. Wan, H. Qu, Y. Han, P. Zheng, H. Zhang, Y. Cao, T. Chen, and Y. K. Zhao, "Rtgs: Real-time 3d gaussian splatting slam via multi-level redundancy reduction," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1838–1851. [Online]. Available: <https://doi.org/10.1145/3725843.3756099>
- [46] D. Wagner, "Motion to photon latency in mobile ar and vr," <https://medium.com/@DAQRI/motion-to-photon-latency-in-mobile-ar-and-vr-99f82c480926>, Aug. 2018, [Online; accessed 2026-04-01].
- [47] VR/AR Wiki, "Motion-to-photon latency," https://vrarwiki.com/wiki/Motion-to-photon_latency, 2026, [Online; last edited 2026-03-16; accessed 2026-04-01].
- [48] Meta, "Missed frames and frame recovery," <https://developers.meta.com/horizon/documentation/native/android/os-missed-frames/>, 2026, [Online; accessed 2026-04-01].
- [49] L. Liu, M. Saed, Y. H. Chou, D. Grigoryan, T. Nowicki, and T. M. Aamodt, "Lumibench: A benchmark suite for hardware ray tracing," in *2023 IEEE International Symposium on Workload Characterization (IISWC)*, 2023, pp. 1–14.
- [50] H. Henriques, E. Oliveira, E. Clua, and D. Trevisan, "A mixed path tracing and nerf approach for optimizing rendering in xr displays," in *Proceedings of the 25th Symposium on Virtual and Augmented Reality*, ser. SVR '23. New York, NY, USA: Association for Computing Machinery, 2024, p. 123–130. [Online]. Available: <https://doi.org/10.1145/3625008.3625027>
- [51] L. Franke, L. Fink, and M. Stamminger, "Vr-splatting: Foveated radiance field rendering via 3d gaussian splatting and neural points," *Proc. ACM Comput. Graph. Interact. Tech.*, vol. 8, no. 1, May 2025. [Online]. Available: <https://doi.org/10.1145/3728302>
- [52] Meta, "Set display refresh rates," <https://developers.meta.com/horizon/documentation/native/android/mobile-display-refresh-rate/>, 2026, [Online; accessed 2026-04-01].
- [53] Meta, "Rendering," <https://developers.meta.com/horizon/documentation/unreal/unreal-advanced-rendering/>, 2026, [Online; accessed 2026-04-01].
- [54] G. Denes, K. Maruszczyk, M. Ashby, G. Wetzstein, K. Myszkowski, and P. Didyk, "Temporal resolution multiplexing: Exploiting the limitations of spatio-temporal vision for more efficient vr rendering," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 5, pp. 2116–2128, 2019.
- [55] A. Podobas, K. Sano, and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE Access*, vol. 8, pp. 146 719–146 743, 2020.
- [56] G. Gobieski, S. Ghosh, M. Heule, T. Mowry, T. Nowatzki, N. Beckmann, and B. Lucia, "Riptide: A programmable, energy-minimal dataflow compiler and architecture," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 546–564.
- [57] G. Gobieski, A. O. Atli, K. Mai, B. Lucia, and N. Beckmann, "Snafu: An ultra-low-power, energy-minimal cgra-generation framework and architecture," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1027–1040.

- [58] Z. Li, D. Wijerathne, and T. Mitra, "Coarse-grained reconfigurable array (cgra)," in *Embedded Computing*. Springer, 2024.
- [59] NVIDIA Corporation, "NVIDIA Jetson Orin NX Series Data Sheet," Online, 2022, dS-10712-001. Accessed: 2026-06-18. [Online]. Available: <https://developer.nvidia.com/downloads/jetson-orin-nx-series-data-sheet>
- [60] N. Corporation, "Jetson Nano Brings the Power of Modern AI to Edge Devices," Online, n.d., accessed: 2026-06-18. [Online]. Available: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/product-development/>
- [61] S.-H. Noh, B. Shin, J. Choi, S. Lee, J. Kung, and Y. Kim, "Flexnerf: A multi-dataflow, adaptive sparsity-aware accelerator for on-device nerf rendering," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1894–1909. [Online]. Available: <https://doi.org/10.1145/3695053.3731107>
- [62] S. Li, C. Li, W. Zhu, B. T. Yu, Y. K. Zhao, C. Wan, H. You, H. Shi, and Y. C. Lin, "Instant-3d: Instant neural radiance field training towards on-device ar/vr 3d reconstruction," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589115>
- [63] C. Reiser, S. Peng, Y. Liao, and A. Geiger, "Kilonerf: Speeding up neural radiance fields with thousands of tiny mlps," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 14 315–14 325.
- [64] A. Gilles, P. Le Gargasson, G. Hocquet, and P. Gioia, "Holographic near-eye display with real-time embedded rendering," in *SIGGRAPH Asia 2023 Conference Papers*, 2023, pp. 1–10.
- [65] Z. Zhang, Z. Li, H. Kim, and C. Liu, "Boxr: Body and head motion optimization framework for extended reality," *arXiv preprint arXiv:2410.13084*, 2024.
- [66] C. Tan, D. Patil, A. Tumeo, G. Weisz, S. Reinhardt, and J. Zhang, "Vecpac: A vectorizable and precision-aware cgrr," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [67] S. Li, K. Chen, J. H. Ahn, J. B. Brockman, and N. P. Jouppi, "Cactip: Architecture-level modeling for sram-based structures with advanced leakage reduction techniques," in *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2011, pp. 694–701.
- [68] S. Sarangi and B. Baas, "Deepscaletool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5.
- [69] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [70] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, "Freepdk: An open-source variation-aware design kit," in *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*, 2007, pp. 173–174.
- [71] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained dram: energy-efficient dram for extreme bandwidth systems," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-50 '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 41–54. [Online]. Available: <https://doi.org/10.1145/3123939.3124545>
- [72] Z. Zhang, H. Wang, S. Han, and W. J. Dally, "Sparch: Efficient architecture for sparse matrix multiplication," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 261–274.
- [73] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110.
- [74] NVIDIA, "Jetson Orin Nano Series, Jetson Orin NX Series and Jetson AGX Orin Series," *NVIDIA Jetson Linux Developer Guide*, 2024, jetson Linux r36.4.4. Accessed: 2026-06-12. [Online]. Available: <https://docs.nvidia.com/jetson/archives/r36.4.4/DeveloperGuide/SD/PlatformPowerAndPerformance/JetsonOrinNanoSeriesJetsonOrinNxSeriesAndJetsonAgxOrinSeries.html>
- [75] Meta, "Meta Quest Pro," <https://www.meta.com/quest/quest-pro/>, 2022, accessed: 2025-07-21.
- [76] Y. Yeshurun, M. Carrasco, and L. T. Maloney, "Bias and sensitivity in two-interval forced choice procedures: Tests of the difference model," *Vision research*, vol. 48, no. 17, pp. 1837–1851, 2008.
- [77] H. Wang, W. Liu, K. Chen, Q. Sun, and S. Q. Zhang, "Process only where you look: Hardware and algorithm co-optimization for efficient gaze-tracked foveated rendering in virtual reality," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 344–358. [Online]. Available: <https://doi.org/10.1145/3695053.3731110>
- [78] J. Qin, T. Xia, C. Tan, J. Zhang, and S. Q. Zhang, "Picachu: Plug-in cgrr handling upcoming nonlinear operations in llms," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 845–861. [Online]. Available: <https://doi.org/10.1145/3676641.3716013>
- [79] T. K. Bandara, D. Wu, R. Juneja, D. Wijerathne, T. Mitra, and L.-S. Peh, "Flex: Introducing flexible execution on cgrr with spatio-temporal vector dataflow," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [80] T. Xia, H. Wang, and S. Q. Zhang, "Segment only where you look: Leveraging human gaze behavior for efficient computer vision applications in augmented reality," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, ser. ASPLOS '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1693–1710. [Online]. Available: <https://doi.org/10.1145/3779212.3790216>
- [81] NVIDIA, "NVIDIA Jetson Thor: Advanced AI for Physical Robotics," <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-thor/>, n.d., accessed: 2026-06-16.
- [82] Meta, "Meta Quest 3: Next-Gen Virtual Reality Headset," <https://www.meta.com/quest/quest-3/>, n.d., accessed: 2026-06-16.
- [83] D. Heaney, "Quest 3 Teardown Reveals It's Mostly Battery Inside," <https://www.uploadvr.com/quest-3-teardown-battery/>, 2023, accessed: 2026-06-16.
- [84] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger, "Mip-Splatting: Alias-Free 3D Gaussian Splatting," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA: IEEE Computer Society, Jun. 2024, pp. 19 447–19 456. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CVPR52733.2024.01839>
- [85] D. Ha, L. Liu, Y. H. Chou, S. Go, W. W. Ro, H.-W. Tseng, and T. M. Aamodt, "Generalizing ray tracing accelerators for tree traversals on gpus," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1041–1057.
- [86] J. Lee, J. Kim, J. Park, and J. Sim, "Vr-pipe: Streamlining hardware graphics pipeline for volume rendering," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 217–230.
- [87] W. Kim, Y. Lee, Y. Kim, J. Hwang, S. Oh, J. Jung, A. Huseynov, W. G. Park, C. H. Park, D. Mahajan, and J. Park, "Pimba: A processing-in-memory acceleration for post-transformer large language model serving," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 292–307. [Online]. Available: <https://doi.org/10.1145/3725843.3756121>
- [88] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589038>
- [89] A. Ramachandran, S. Kundu, and T. Krishna, "Microscopiq: Accelerating foundational models through outlier-aware microscaling quantization," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1193–1209. [Online]. Available: <https://doi.org/10.1145/3695053.3730989>